

DRÆM - Distributed Ray- Accelerated Engine through MIMD

Alexander Gallagher
alexg5@illinois.edu
UIUC

Saipranav Venkatakrisnan
sv34@illinois.edu
UIUC

Ryan Horstman
ryanjh4@illinois.edu
UIUC

Abstract

Ray tracing on modern GPUs suffers from three structural inefficiencies rooted in the SIMT execution model: warp divergence from per-ray BVH traversal paths, non-coalesced DRAM access from pointer-chasing tree traversal, and idle threads caused by load imbalance across rays. These are not fixable by hardware additions like RT Cores—they are consequences of forcing an inherently divergent, irregular workload onto a lockstep, bulk-synchronous machine.

DRÆM (Distributed Ray-Accelerated Engine through MIMD) is a proposed alternative architecture that replaces the GPU’s branch-divergence problem with an explicit NoC routing problem. Rather than grouping threads into warps and stalling on divergence, DRÆM assigns BVH geometry to core-local SRAM, routes rays as messages across a 2D mesh NoC, and processes each ray independently on whatever core owns the relevant geometry. This eliminates warp divergence by construction, removes DRAM from the traversal critical path, and converts load imbalance into a distributed scheduling problem amenable to dynamic core cloning.

We describe the architecture, implement a cycle-accurate Rust simulator, and report preliminary metrics from 150K cycles of simulation. Our results confirm the expected NoC contention pattern as hot geometry cores attract high ray traffic, validating the core architectural trade-offs and motivating future work on load balancing and per-stack caching.

Source Code: <https://github.com/alexandergdot123/RustSim>

1 Introduction

Modern GPUs are powerful general-purpose parallel processors, but ray tracing exposes their architectural limits. The fundamental problem is not a lack of hardware investment, as NVIDIA has dedicated fixed-function RT Cores to BVH traversal since the Turing architecture, but rather that the GPU’s SIMT execution model is structurally incompatible with ray tracing’s workload characteristics. Rays from different pixels diverge immediately: they take different paths through the BVH, hit different geometry, spawn different secondary rays, and complete at different times. A GPU warp executing in lockstep must mask inactive threads while the slowest thread finishes, wasting execution slots on every bounce. RT Cores offload the intersection math but do not fix the divergence or memory access patterns of the threads requesting those intersections.

This paper identifies three root causes of GPU ray tracing inefficiency and proposes DRÆM as a targeted architectural response to each:

- **Branch divergence:** On a GPU, threads in a warp must execute the same instruction. Divergent BVH traversal paths cause warps to serialize. DRÆM replaces warps with independent per-ray message routing: each ray is a message

forwarded hop-by-hop through the NoC to whichever core owns the relevant BVH subtree. There is no concept of a warp and therefore no warp divergence.

- **DRAM latency and non-coalesced access:** BVH traversal is a pointer-chasing workload. GPU caches, designed for spatial and temporal locality, underperform; DRAM efficiency averages only 46% on ray tracing workloads [5]. DRÆM pins each BVH subtree to a core’s private 48 KB SRAM, removing DRAM from the traversal critical path entirely. Every intersection test reads from on-chip memory.
- **Load imbalance:** Some geometry is intersected by many more rays than others, overloading some threads while others are idle. On a GPU this manifests as warp occupancy loss; on DRÆM it manifests as NoC congestion on hot cores. The difference is that DRÆM can respond dynamically: idle cores can be cloned with a busy core’s firmware and geometry, distributing the load across the mesh.

DRÆM is not intended as a direct performance competition against GPUs running fixed-function RT Cores. Rather, it is an existence proof that the three pathologies above are avoidable by design. We describe the architecture, its implementation in a cycle-accurate simulator built with the Rust Programming Language, and preliminary simulation results from 150K cycles that confirm the predicted NoC contention pattern and motivate future work on load balancing.

2 Background and Motivation

Ray tracing simulates physically accurate lighting and rendering by casting rays from a virtual camera into a scene and recursively evaluating reflections, refractions, and shadow interactions. Because testing each ray against every primitive in the scene is computationally intense, rays run intersections against a Bounding Volume Hierarchy; a tree of axis-aligned bounding boxes that spatially partitions scene geometry, allowing large portions of the scene to be culled with inexpensive ray-box tests before the costly Möller-Trumbore ray-triangle intersections are performed. NVIDIA’s Turing architecture (2018) introduced dedicated RT Cores to accelerate precisely this operation, offloading BVH traversal and intersection from general-purpose CUDA cores into fixed-function hardware [9]. Despite this hardware investment, ray tracing remains a demanding workloads in graphics, and RT Core acceleration has not resolved the fundamental performance bottlenecks of the GPU execution model. The root cause of these bottlenecks is the structural mismatch between ray tracing’s execution characteristics and the GPU’s SIMT model. Rays originating from different pixels take entirely different paths through the BVH, causing threads within a warp to diverge dramatically, worsening with each ray’s bounce throughout the scene. Vulkan-Sim [5], a cycle-accurate GPU simulator for ray tracing, quantifies this mismatch: SIMT efficiency in RT units averages only 35%, dropping as low as 7% for path tracing

workloads with incoherent secondary rays, with threads sitting idle for 59% of cycles waiting for other threads to complete long traversal paths. The random, pointer-chasing memory access patterns of BVH traversal compound this problem by undermining the GPU's cache hierarchy, which is designed around the spatial and temporal locality. DRAM efficiency averages only 46%, and L1 data cache hit rates fall below 50% in several workloads as divergent rays scatter across unrelated BVH subtrees, thrashing cache lines that conventional compute workloads would reuse [5]. RayN [6] further isolates memory latency, not bandwidth, as the primary bottleneck, with near-memory computation achieving a 3.0 $\times$ speedup and 70% energy reduction by eliminating GPU-to-DRAM round-trip latency. Taken together, these findings motivate a fundamentally different approach: rather than improving GPU cache behavior or branch-divergence characteristics within the SIMT model, we propose eliminating the shared memory hierarchy from the BVH traversal path entirely, pinning geometry to core-local SRAM and routing rays through the mesh via the NoC.

3 Design

Our architecture consists of a two-dimensional mesh of simple, deeply multithreaded cores, each equipped with 48 KB of private on-chip SRAM. The mesh is organized as a grid of stacks, where each stack comprises a tile of cores sharing a local DRAM bank, connected to neighboring stacks via inter-stack channels. Cores within the mesh are connected by a bidirectional 2D NoC using XY routing. Each core supports hardware thread contexts, allowing it to switch between in-flight rays to hide NoC and DRAM latency.

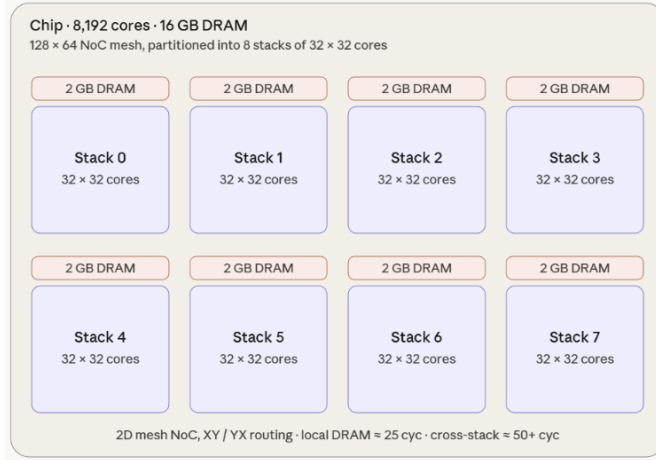


Figure 1: Chip Architecture Diagram

BVH Partitioning: Before rendering begins, the BVH is partitioned using a depth-first traversal that greedily accumulates nodes into subtrees bounded by the per-core SRAM capacity. The memory footprint of each subtree accounts for BVH node data (bounding boxes and child pointers), triangle index lists, material pointers, and the unique vertex positions referenced by each triangle. Unique vertices are tracked using a fixed-point spatial hash to avoid double-counting shared geometry. When a candidate subtree would exceed the SRAM budget, it is cut at that node boundary, and the root of the

excluded child becomes the entry point of a new subtree assigned to a different core. This partitioning produces two classes of cores: leaf cores, which own BVH subtrees that terminate in triangles and perform both AABB slab tests and Möller-Trumbore intersection, and branch cores, which own upper levels of the hierarchy and perform traversal routing, forwarding incoming rays toward the correct child leaf cores via the NoC.

Ray Routing: During rendering, each ray is represented as a message carrying its origin, direction, and traversal state. A ray enters the mesh at the core owning the BVH root and is forwarded hop-by-hop through the NoC towards whichever core owns the next relevant subtree. At each hop, the receiving core performs its local intersection tests entirely from SRAM and either produces a hit result or forwards the ray onward.

Core ISA and Pipeline: Each core implements a custom 32-bit ISA supporting integer ALU operations, floating-point computation in FP32, FP16, and FP8 (E4M3) formats, SRAM loads and stores, DRAM reads and writes (both local and remote), atomic addition for synchronization, and a SendFlit instruction for NoC message sending. Long-latency operations, such as floating-point arithmetic, multiplication, division, and DRAM accesses, are pipelined through scoreboard-tracked queues, and the core switches to other hardware thread contexts while waiting. Local DRAM accesses carry lower latency in comparison to cross-stack DRAM accesses. The pipeline is two-stage (fetch and execute), with branch hints embedded in the instruction encoding to reduce flush penalties on predictable control flow.

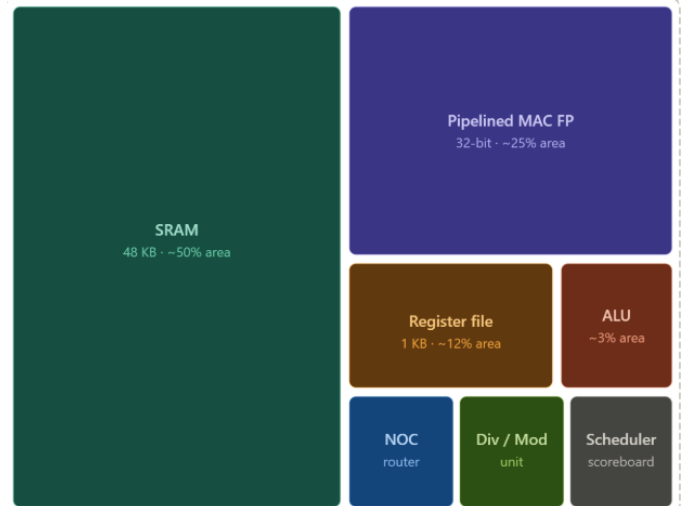


Figure 2: Core Architecture Diagram

Simulation Infrastructure: Our simulator, implemented in Rust, runs each stack as an independent OS thread, synchronized via a cycle barrier. Per-core telemetry, including NoC utilization and congestion in each direction, mailbox backpressure, core busy fraction, and bytes read and written to local and remote DRAM, is recorded per epoch and exported for visualization and analysis.

Comparison Methodology and Area Model: We compare DRAM against an RTX 3090 with RT Cores disabled on the GPU and

no fixed-function hardware on DRÆM, ensuring the comparison reflects architectural behavior compute rather than fixed-function acceleration. This isolates the question of whether a MIMD mesh with SRAM-pinned geometry can outperform a SIMT architecture on BVH traversal using equivalent classes of programmable logic. To estimate the die area of a DRÆM core, we model each core after the Cerebras wafer-scale engine core on TSMC N7, which is comparable in density to Samsung 8nm, the process used for GA102. A Cerebras core devotes approximately half of its 0.05 mm^2 size to 48 KB of SRAM.

The proposed remaining logic per core is minimal: a small FP pipeline; an ALU; a register file; a fire-and-forget DRAM request sender and response receiver; a scoreboard and context selector for hardware thread management; and a mailbox and NoC interface. At 8,192 cores, the total estimated die area for the DRÆM compute array is approximately 409 mm^2 . In contrast, prior die examinations estimate that approximately 60% of the 628 mm^2 GA102 die is allocated to SM and L1 cache logic, yielding roughly 377 mm^2 of programmable compute area—a difference of only 8% from DRÆM’s estimated compute footprint, providing a meaningful basis for architectural comparison.

4 Implementation

GPU Baseline Development:

The GPU baseline is implemented in CUDA, adapted from existing open-source BVH traversal kernels, and evaluated via full PTX trace execution in AccelSim against an RTX 3090-equivalent configuration. All tests and data are collected off of the same selected scene, camera position, camera angle, and lighting algorithm. The colored (non-raytraced) base image is seen in Figure 3.

In order to accurately compare our custom architecture to current SOTA path tracing in software, we optimized our CUDA algorithm according to the standard literature so that we compare our current algorithm to the largest suite of optimizations available to the NVIDIA SIMT architecture. During the pre-processing stage, after transforming the input scene into a BVH tree using tinyBVH, we collapsed the tree into a BVH-4, where each internal node holds up to four children, increasing spatial locality during node fetches for better cache performance. During traversal, shadow rays use any-hit early exit, returning on the first intersection found and often pruning the majority of the subtree. CUDA-level optimizations include separate kernels for primary ray tracing (kTrace) and shadow evaluation (kShadow), and ray sorting to reduce artificial branch divergence before each shadow pass. Ray sorting improves cache hit rates by grouping spatially nearby shadow rays together, increasing the probability that rays traverse the same path down the BVH tree simultaneously. Finally, we use a deterministic pseudorandom number generator for ray bounce calculations. This eliminates state within threads and allows runs to be replicable, maintaining consistent baseline data.

After confirming functionality of the CUDA on a personal RTX 3060 Laptop GPU (output in Figure 4), we profiled the full-resolution kernel (2560×1440 , spp=1) using NVIDIA Nsight Compute to characterize real hardware behavior. To obtain more detailed microarchitectural statistics, we also ported the CUDA into AccelSim’s execution-driven GPGPU-Sim mode and adjusted the provided

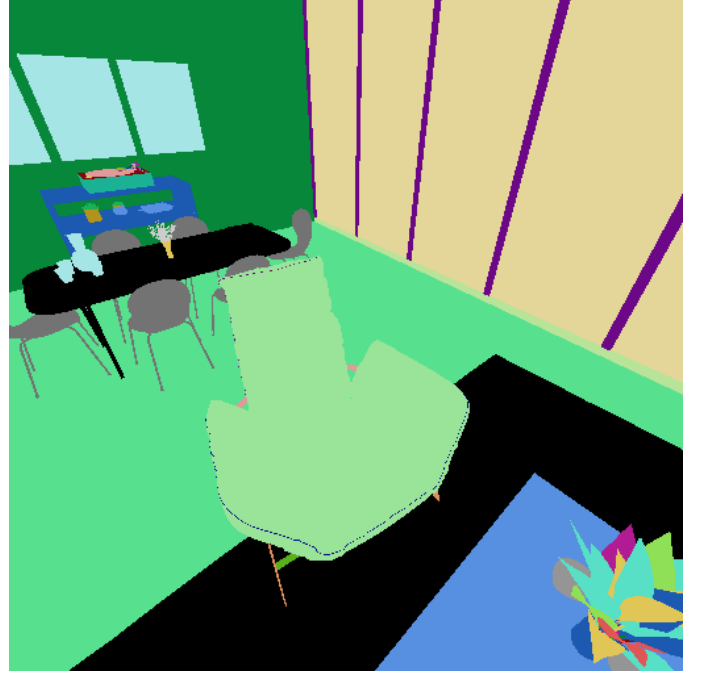


Figure 3: Colored Image (non-ray traced) from CUDA (256x256 resolution)

RTX 3070 configuration to match the RTX 3090’s 82-SM GA102 die. Because AccelSim simulates every pipeline stage cycle-accurately, simulation throughput is on the order of thousands of cycles per second of wall time, making full-resolution runs impractical. We therefore ran a 32×32 image at spp=1 to collect initial statistics, preserving all other scene parameters: four bounces (one primary and three secondary) and three shadow rays per bounce, with three colored point lights (red, green, blue) in the room. Due to the small number of pixels, an image of this size does not fully utilize the GPU’s SMs, so we then ran AccelSim for a longer period at a larger resolution, specifically 256×256 , to fully saturate the SM array and produce baseline statistics that better reflect the GPU’s memory subsystem under load. The result was 57.260 million cycles at 1.132 GHz, and a utilization of 86 SMs. Upon analysis of AccelSim’s produced statistics it was evident that there were 28.7 percent of the cycles in which no warps could execute due to waiting on memory accesses. The memory bound nature of the CUDA implementation was confirmed as previously established.

Custom Simulator Implementation: The DRÆM system is implemented and evaluated via a custom cycle-accurate simulator written in Rust. The simulator organizes cores into a two-level hierarchy: a global mesh of cores subdivided into *stacks*, where each stack is a tile of cores sharing a local DRAM bank. Each stack runs in the simulator as an independent OS thread, and all stacks synchronize via a global cycle barrier, ensuring that every stack advances in lockstep. Within each cycle, every core in the stack executes one step of its instruction pipeline via a tick call, after which the stack services any pending cross-stack DRAM requests and forwards any returned results to the appropriate cores.

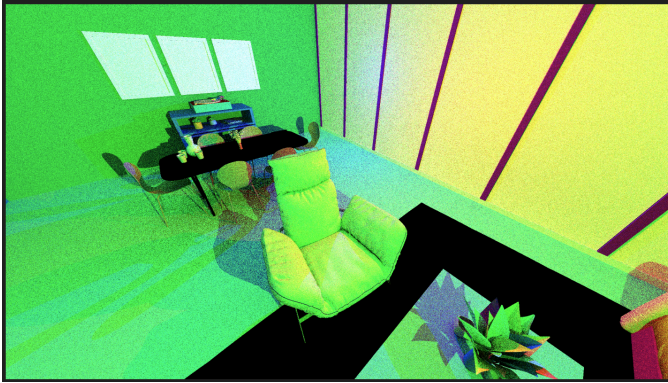


Figure 4: Ray-Traced Image from optimized CUDA (2560x1440, spp = 1)

Each core models a scoreboard-tracked register file across all hardware thread contexts. On each cycle, the core attempts to execute the decoded instruction for the currently active context. If the instruction is a long-latency operation—such as a DRAM load, atomic add, or NoC send—the result is placed into a pending pipeline queue tagged with the cycle at which it will be available, and the destination register is marked not ready. If the output NoC buffer is full or a required register is not yet ready, the instruction is squashed and the program counter is not advanced; the core switches to another runnable context and the instruction is re-fetched when that context is next selected. Local DRAM accesses are resolved directly within the stack thread at a lower latency, while cross-stack DRAM requests are dispatched as messages to the owning stack’s OS thread and return after a longer modeled latency. NoC flits are injected into the output buffer with XY routing, and the simulator tracks the Manhattan distance traversed by each flit for telemetry purposes.

Per-core telemetry is recorded each epoch and includes NoC utilization and congestion in all four directions, mailbox backpressure, core busy fraction, bytes read and written to both local and remote DRAM, flits sent and received, and the total Manhattan distance traversed by sent and received flits. At simulation end, all telemetry is exported as NumPy arrays indexed by time, stack, and core position, enabling spatial and temporal visualization of NoC hotspots, DRAM pressure, and core utilization across the mesh.

Core Firmware: Each DRAM core is equipped with 48 KB of private on-chip SRAM. Cores are assigned one of two roles: leaf cores, which own terminal BVH subtrees and perform Möller-Trumbore ray-triangle intersection entirely from SRAM, and branch cores, which own internal BVH levels and route rays toward child cores by forwarding them as messages over the NoC. All branch cores share the uppermost levels of the hierarchy connecting all other branch cores together, serving as the entry point for all rays entering the mesh. When a ray completes traversal of a core’s local treelet, it is packaged as a 16-flit NoC message and sent directly to whichever core owns the next relevant subtree. BVH traversal thus proceeds as a sequence of NoC hops across the mesh, with each hop performing intersection tests entirely from SRAM, eliminating off-chip DRAM from the traversal path entirely.

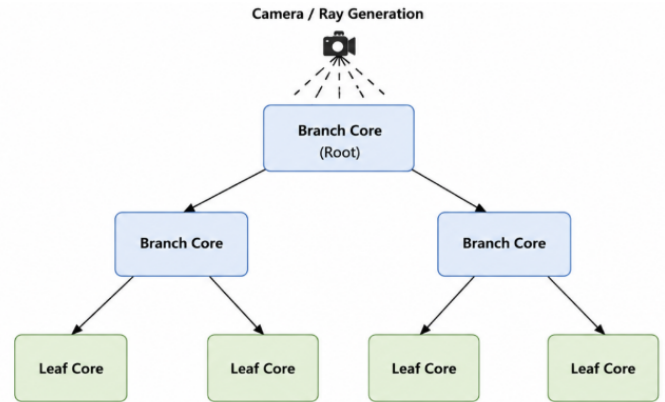


Figure 5: Abstract Model of the Core Role Assignment

When a core receives a ray destined for its local treelet, it enqueues the ray either in a small SRAM-resident queue for immediate processing or, when SRAM capacity is exceeded, in a larger DRAM-backed queue. However, the presence of queues alone does not eliminate deadlocks. Because branch cores both receive rays from upstream and forward rays downstream to their owned leaf cores, cycles can form in the ray flow graph: a branch core’s incoming queue fills waiting for a leaf core to drain, while that leaf core is simultaneously waiting to forward a completed ray back through the branch core, preventing either from making progress. To break this cycle, each leaf core stores pointers to all neighboring nodes within its branch core’s subtree, allowing it to forward a ray directly to a sibling leaf core without re-entering the branch core. Additionally, each branch core’s thread contexts are virtualized into two halves, with separate queues for rays entering from the broader mesh and rays returning from owned leaf cores. Each virtualized half is entered exactly once per ray, converting the ray flow graph into a DAG and eliminating circular queue dependencies by construction. Without this algorithm, a ray traversing every node in a branch core’s subtree would need to pass through the branch core n times, where n is the number of owned leaf cores, creating n opportunities for circular dependencies to form. A consequence of this design is that rays cannot be reordered mid-subtree based on intersection distance, and so front-to-back traversal within a branch core’s subtree is not implemented.

At any given time, zero, one, or more cores may serve as the representative for a particular treelet. This mapping is not static: the number of cores representing a treelet can grow or shrink dynamically in response to load. Each node in the BVH maintains a DRAM-resident list of core IDs currently representing it, which serves as the source of truth for treelet ownership. When a core forwards a ray to a node, it consults a locally cached copy of that node’s representative core ID rather than reading from DRAM on every forward. This cached value is periodically refreshed from the DRAM-resident list using a randomized index to distribute load across multiple representatives without requiring coordination.

Each core monitors its own utilization over a rolling window of cycles. A core that falls below a forwarding rate threshold marks itself as idle and inserts its core ID into a spatially-organized tree

of idle queues backed by DRAM. When a core determines it is too busy—identified by its DRAM ray queue filling beyond a threshold—it searches this idle queue tree for a candidate core to clone itself onto. The search traverses the tree bottom-up from the busy core's own position, prioritizing candidates that are physically nearby on the mesh to minimize the NoC latency of transferring firmware, geometry, and future rays. Once a candidate is found, the busy core transmits its firmware and geometry to the idle core over the NoC, after which the new core registers itself in the DRAM-resident representative list for that treelet and begins draining rays.

5 Execution Summary

The full DRÆM toolchain was completed across three phases. First, a BVH preprocessing pipeline was built to import OBJ scenes, construct and partition the hierarchy across the 8,192-core mesh, and serialize all triangle, vertex, and node data into the simulator's DRAM model. The benchmark scene contains 1.9 million triangles. Second, a custom assembler was implemented to translate the project's hand-written ISA assembly into executable bytecode loaded directly into the simulator. The firmware comprises approximately 4,500 lines of hand-written assembly across two roles: leaf-core firmware handling Möller-Trumbore intersection, and branch-core firmware handling AABB traversal and ray forwarding. Both programs cover the full ray lifecycle, including BVH download, traversal, ray forwarding, shadow and bounce ray spawning, and pixel accumulation. Third, the GPU baseline was finalized. The optimized CUDA kernels were validated on a physical RTX 3060 Laptop GPU and profiled with NVIDIA Nsight Compute. AccelSim runs were conducted at 32×32 and 256×256 resolutions against an RTX 3090-class configuration; the 256×256 run produced 57.260 million cycles at 1.132 GHz with 86 active SMs, confirming memory-bound behavior with 28.7T. The primary execution risk materialized as expected: hand-written assembly correctness was the most time-consuming milestone, with some features like load-balancing requiring more time than was available. For this reason, we leave this area of exploration to future work.

6 Evaluation

To characterize the GPU baseline, we profiled the optimized CUDA kernels on a physical RTX 3060 Laptop GPU at 2560×1440 resolution using NVIDIA Nsight Compute. The workload was classified as high-throughput, sustaining 90.57% compute and memory throughput simultaneously over 300,841,787 elapsed cycles at 1.30 GHz. The LSU pipeline was the most heavily utilized at 90.6% of executed instructions, consistent with the memory-bound nature of BVH traversal. Thread divergence was significant: the average active thread count per warp was 17.66 of 32, with Nsight estimating a 41.9% potential speedup from eliminating divergence alone. Global load accesses utilized only 4.0 of 32 bytes per transmitted cache sector, producing over 2 billion excessive sectors (23% of all global traffic) confirming the non-coalesced, pointer-chasing access pattern inherent to BVH traversal.

These results were collected on an RTX 3060 Laptop GPU (30 SMs). To extrapolate to an RTX 3090 (82 SMs), we scale throughput by the SM ratio under the assumption that the workload is sufficiently parallel to utilize the additional SMs at equivalent occupancy,

which is a reasonable assumption at resolutions large enough to fully saturate the SM array. We acknowledge this is an approximation; more precise RTX 3090-class figures will be obtained from AccelSim once longer simulation runs at larger resolutions complete. The 32×32 AccelSim run confirms the qualitative bottleneck profile but underutilizes the simulated SM array due to the small pixel count, and we intend to run at 128×128 or larger to produce fully saturated baseline statistics.

6.1 DRÆM Simulator Status

The DRÆM software and simulation stack is now largely operational across the full toolchain. A custom assembler has been implemented to translate the project's custom ISA assembly into executable bytecode, which is loaded directly into the simulator DRAM model. The simulator currently boots and executes three hand-written assembly programs: a bootloader, leaf-core firmware, and branch-core firmware. The cycle-accurate multithreaded Rust simulator has been developed to model the complete DRÆM architecture, including the full 2D mesh of 8,192 cores, distributed DRAM stacks, NoC links with configurable latency, hardware thread context switching, and per-context register readiness tracking. The simulator also exports detailed telemetry, including per-core utilization, network congestion, and execution statistics as NumPy-compatible arrays for later visualization and analysis. To support scene ingestion and memory initialization, a BVH preprocessing pipeline was built that imports OBJ scenes, constructs a BVH hierarchy, partitions nodes spatially across the simulated core mesh, and serializes all triangle, vertex, and BVH node data into the simulator DRAM model. Core initialization and SRAM population are now functioning correctly, and the simulator has successfully begun testing ray spawning and BVH traversal using AABB intersection tests. The leaf-core and branch-core firmware pseudocode has also been fully designed and covers the entire ray lifecycle, including BVH downloads, traversal, ray forwarding, shadow and bounce ray spawning, pixel accumulation, and dynamic role switching between traversal and intersection workloads.

6.2 DRÆM Preliminary Metrics

Due to time constraints and the scope of the course, all DRÆM metrics reported here were collected over **150,000 simulation cycles**. In this window, 15 rays completed end-to-end, averaging approximately 10,000 cycles per ray. These figures reflect early-stage ray-tracing execution and are expected to improve substantially as the code matures and longer runs are performed.

AABB Intersection Activity: Branch cores performed **48,238 AABB tests** over the 150K cycle window. The per-core test rate varied widely: cores assigned to geometry that intersects few rays performed between 0 and approximately 15 tests, while cores assigned to highly traversed geometry performed up to **300 AABB tests on average** over the same window. This spread directly reflects the load imbalance predicted by our architecture: certain subtrees attract a disproportionate share of ray traffic, overwhelming the cores that own them while leaving others idle.

Memory Traffic: Local DRAM (per-stack, referred to as the "close" stack) accounted for **123,039,836 bytes** of total traffic, while far (cross-stack) DRAM accounted for **176,123,762 bytes**. The fact

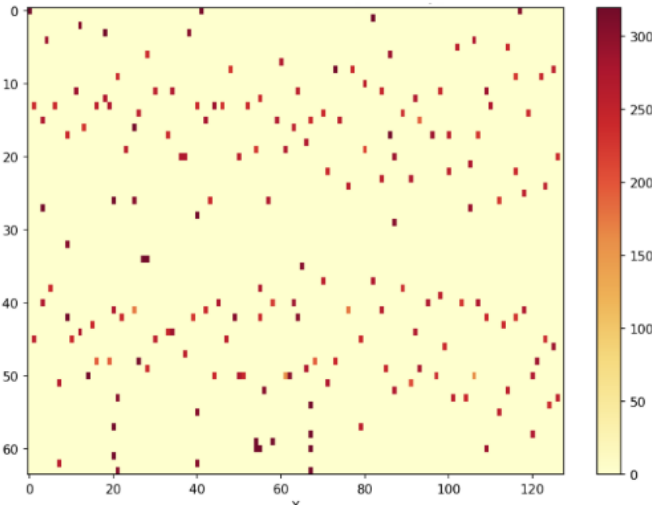


Figure 6: Heat Map Showing AABB Execution Distribution

that far traffic exceeds close traffic is consistent with the early simulation phase, in which rays are being routed across many stacks as the mesh initializes geometry. The dominant contributor to close-stack reads during this window is **stack 0**, which carries the SRAM initialization sequence: every core must download its BVH subtree from DRAM into local SRAM before processing any rays. This initialization traffic is a one-time cost; as simulation cycles accumulate, stack 0 reads are expected to decrease dramatically as no cores remain in the initialization phase and all SRAM contents are already warm.

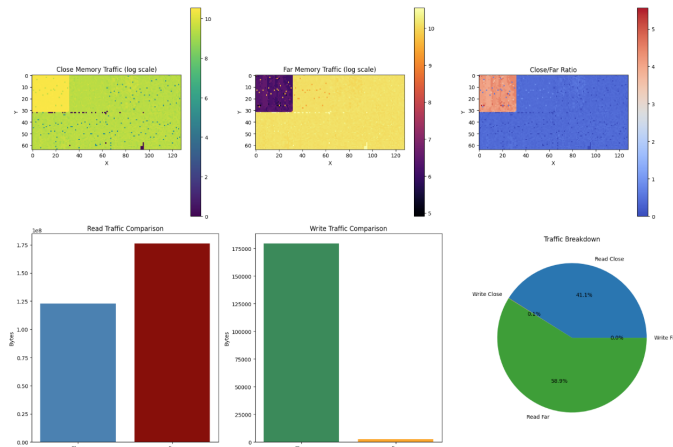


Figure 7: Memory Traffic Visualizations

Core Behavior Over Time:

- *Early phase (initialization)*: All cores are busy loading BVH geometry from DRAM into local SRAM. Work is approximately evenly distributed across the mesh, as each core independently initializes its own assigned subtree.

- *Mid-simulation*: Cores complete initialization and begin processing rays. Branch cores start performing AABB tests and forwarding rays toward leaf cores. Load distribution becomes uneven: cores assigned to geometry that intersects many camera rays begin receiving high ray traffic via the NoC, while neighboring cores with less-traversed geometry become relatively idle. NoC contention begins concentrating around these hot cores.
- *Late phase*: A small subset of cores accounts for the majority of AABB tests and ray forwarding activity. Many other cores are idle or merely routing rays through toward the hot cores. The mesh exhibits high NoC congestion on the links adjacent to these hot cores, while most of the mesh is underutilized. This is the manifestation of the load imbalance problem DRÆM’s dynamic core cloning mechanism is designed to address; the metrics here confirm that cloning is necessary in practice, not just in theory.

7 Discussion

The preliminary results confirm the central architectural bet behind DRÆM: replacing branch divergence with NoC routing does not eliminate imbalance—it relocates it to a place where the architecture can respond. On a GPU, a warp stalled on a divergent thread cannot help other warps; the imbalance is invisible to the scheduler at the right granularity. On DRÆM, imbalanced load is visible as a filled ray queue on a specific core, which triggers cloning. The 150K-cycle window shows exactly this pattern emerging: a small number of cores accumulate AABB tests while most of the mesh goes idle.

This is an encouraging result, not a failure. It means the architecture is behaving as designed and that dynamic cloning—the mechanism built to address exactly this situation—will have real work to do. Future longer-run simulations with cloning enabled will determine whether the cloning algorithm is fast enough to prevent prolonged hot-core saturation.

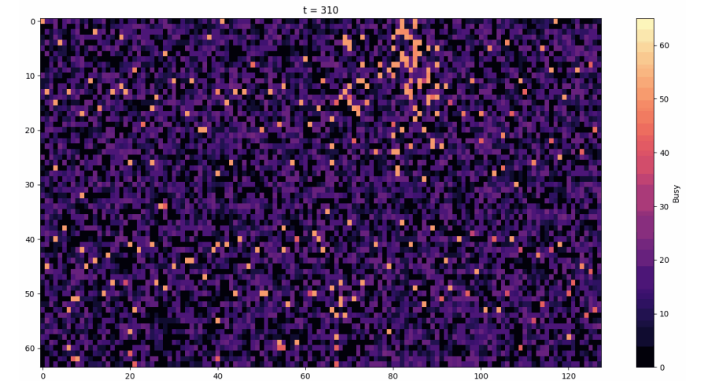


Figure 8: Heat Map of NOC Utilization Highlighting the Work Imbalance

8 Related work

Vulkan-Sim: Quantifying GPU Ray Tracing Inefficiencies Vulkan-Sim [5] is a cycle-level GPU architecture simulator for ray tracing

built on GPGPU-Sim. Their evaluation reveals that SIMT efficiency in RT units averages only 35%, dropping as low as 7% for path tracing with incoherent secondary rays. Threads idle for 59% of cycles while waiting for trailing threads to complete traversal, and DRAM efficiency averages only 46% due to address-random BVH accesses. L1 data cache hit rates fall below 50% in several workloads as divergent rays scatter across unrelated BVH subtrees, thrashing cache lines that a traditional compute workload would reuse. These findings directly validate our two core claims: that the SIMT model is fundamentally mismatched to ray tracing's divergent control flow, and that the GPU's cache hierarchy, designed around spatial and temporal locality assumptions, underperform for the pointer-chasing, random-access patterns inherent to tree traversal.

TRaX and STRaTA: MIMD Ray Tracing Architectures The most directly related prior work is TRaX [3] and its successor STRaTA [2]. TRaX is a custom manycore architecture where simple in-order thread processors share floating-point units and L1 caches within thread multiprocessors. The key insight is that ray tracing's divergent control flow maps better to MIMD than SIMD, which is the same with our architecture. Our design, however, differs in three different ways: a 2D mesh NOC topology rather than hierarchical cache organization; a hierarchical BVH ownership scheme distributing geometry across core-local SRAMs; and dynamic core role switching between traversal and shading, whereas TRaX cores are homogeneous and statically assigned.

Aila and Karras: Incoherent Ray Scheduling and Treelets Aila and Karras [1] propose a massively parallel hardware architecture addressing the memory bandwidth bottleneck for incoherent secondary rays. Their architecture centers on hierarchical treelet subdivision: the BVH is partitioned into subtrees (treelets) whose memory footprint approximately matches the L1 cache capacity, and rays are queued at treelet boundaries rather than immediately traversed. When a treelet is finally scheduled for processing, all queued rays sharing that subtree are batched together, amortizing the cost of loading BVH nodes into cache across many rays and reducing DRAM traffic by up to 90% in difficult organic scenes. A stack-top cache and queue-bypassing mechanism further reduce overhead for rays that remain within the same treelet across multiple traversal steps. Our architecture addresses the same problem through a fundamentally different mechanism: rather than scheduling which rays access which BVH data, we statically assign geometry subtrees to specific cores in a 2D mesh, so each ray is physically routed to the core owning the relevant geometry. This converts what Aila and Karras treat as a cache-miss problem into a NOC routing problem.

CoopRT and RayFlex CoopRT [8] modifies the RT unit's warp buffer to redistribute BVH traversal work from longer-running threads to idle threads within the same warp, improving traversal throughput by reducing the time warps spend stalled on stragglers. While targeting the same inefficiency, CoopRT improves load balance within the SIMT model rather than replacing it; our MIMD architecture makes intra-warp load imbalance structurally impossible, since each core independently processes its own ray with no lockstep constraint. RayFlex [4] provides an open-source RTL implementation of the RT unit datapath in Chisel, implementing an 11-stage elastic pipeline for ray-box and ray-triangle intersection validated against the AMD RDNA3 ISA specification. Our per-core

AABB slab tests and Möller-Trumbore routines are software implementations of the same operations RayFlex realizes in hardware.

RayN and AQB8: Memory-Centric Optimizations RayN [6] places RT units in the logic layer of 3D-stacked HBM, achieving 3.0× speedup and 70% energy reduction. Their key finding led us to the realization that memory latency, not bandwidth, is the primary bottleneck. Both RayN and our architecture eliminate the GPU-to-DRAM latency bottleneck, but through opposite mechanisms: RayN moves compute closer to DRAM, while we pin geometry to core-local on-chip SRAM. AQB8 [7] introduces multi-level BVH quantization, storing bounding boxes as 8-bit integers relative to FP32 anchors, achieving 70% DRAM reduction. AQB8's quantization demonstrates that improving memory bandwidth of AABB nodes accesses is critical to improving the performance of tree traversal and provides another direction for potential additional and exploration for our simulator.

AccelSim and Evaluation Methodology AccelSim [9] is a detailed GPU simulation framework supporting trace-driven and execution-driven modes. We leverage AccelSim to establish our GPU baseline, running identical ray tracing workloads through a 3090-class GPU model to provide a rigorous architectural comparison against our manycore MIMD simulator.

9 Conclusion and/or Future Work

A natural extension of this work would be evaluating DRÆM against fixed-function ray tracing hardware such as NVIDIA RT Cores and AMD Ray Accelerators. While this paper disables fixed-function acceleration on the GPU side to isolate architectural behavior of programmable compute, the more commercially relevant question is whether a fully programmable MIMD mesh can approach the efficiency of purpose-built traversal hardware. Fixed-function RT units achieve their performance through deeply pipelined, area-optimized datapaths for AABB slab tests and Möller-Trumbore intersection that amortize control overhead across many concurrent rays in hardware. DRÆM performs the same operations in software on simple cores, but offsets this disadvantage by eliminating the DRAM latency and warp divergence penalties that fixed-function hardware on a SIMT GPU still suffers from. Whether this tradeoff favors DRÆM at scale is an open question, and one that a direct cycle-accurate comparison against an RT-Core-enabled AccelSim configuration or a Vulkan-Sim baseline would be well-positioned to answer.

Importantly, DRÆM's programmable nature does not preclude fixed-function acceleration entirely: each core could in principle be paired with memory-mapped fixed-function units for AABB intersection and Möller-Trumbore evaluation, with software issuing intersection requests by writing ray and geometry data to dedicated registers and reading back results through the same memory interface. This would allow DRÆM to adopt fixed-function acceleration incrementally without sacrificing its programmable MIMD execution model or the SRAM-pinning approach that eliminates DRAM from the traversal path.

References

- [1] T. Aila and T. Karras. 2010. Architecture Considerations for Tracing Incoherent Rays. *Proc. High-Performance Graphics (HPG)*.

- [2] D. Kopta et al. 2013. An Energy and Bandwidth Efficient Ray Tracing Architecture. Proc. High-Performance Graphics (HPG).
- [3] J. Spjut et al. 2009. TRaX: A Multi-Threaded Architecture for Real-Time Ray Tracing. Proc. IEEE Symp. on Application Specific Processors (SASP).
- [4] J. Shen et al. 2025. RayFlex: An Open-Source RT Unit Datapath in Chisel.
- [5] S. Saed et al. 2022. Vulkan-Sim: A GPU Architecture Simulator for Ray Tracing. arXiv:2203.09098 [cs.AR] <https://arxiv.org/abs/2203.09098>
- [6] S. Saed et al. 2025. RayN: Near-Memory Ray Tracing. Proc. ACM/IEEE Int. Symp. on Computer Architecture (ISCA).
- [7] Y. Huang et al. 2025. AQB8: Hardware Accelerator for Quantized BVH Ray Tracing.
- [8] Y. Lee et al. 2024. CoopRT: Cooperative BVH Traversal for Energy-Efficient Ray Tracing. IEEE Computer Architecture Letters (CAL).
- [9] NVIDIA Corporation. 2018. NVIDIA Turing GPU Architecture. <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>